

Abstract Data Types vs Data Structures



The Cinema Analogy

imagine a cinema on **release day**, customers are growing by the minute, the cinema is making millions

yet no one waits more than five minutes, every person is served by hand, and every customer is welcomed by an usher

how can these few employees serve all the increasing number of customers?



The Menu

let's list the tasks we **expect** the employees to do for us:

book a seat, serve us popcorn or a drink, show us our seat

this is a list of things we expect them to do, a **menu**



What vs How

if we have our popcorn, our drink, our tickets, and we're ready to go, why should we care **how** they do it?

the menu is **what** we expect,
the underlying work is **how** they deliver it

the manager can redistribute the roles, i.e., change the implementation at any time, if the menu stays the same



Abstract Data Types

abstract data types are **models, definitions, logical descriptions** from the point of view of the user

we define the **operations** we can perform on given data, and **what** is the expected behavior of these operations



Data Structures

data structures specify **how** the operations defined in the abstract data type will exhibit their behaviors

they are the **concrete implementations** of the abstract data types



The List ADT

the list ADT defines these allowed operations:

add - adds an item to the list

remove - removes an item from the list

get - returns the value at a given index

but was it implemented using an **array** or a **linked list**?
from the ADT alone, we can't know



The Queue ADT

the allowed operations are to **enqueue** or **dequeue** an item in a **first-in-first-out** manner

dequeue returns the item on top of the queue,
enqueue adds the item to the back

what underlying data structure is used?
that is up to the implementer to decide



What About Trees?

some refer to trees as ADTs, others as data structures

when the same word represents both, it most
often represents the **abstract data type**

a tree is in fact an ADT, and the tree
ADT has many different implementations



Summary

abstract data types state **what** are the expected behaviors from a user point of view

they are the **interfaces** for a certain underlying implementation or data structure

these **implementations** state **how** these behaviors will be exhibited without affecting the desired expected outcome

